

Testing Tiap Layer Secara Terpisah

ML Engineer Journey | Phase 3 — Docker & Clean Architecture | 22 Mei 2026

1. Mengapa Testing Penting di Layered Architecture

Setelah refactor penuh ke layered architecture (W11D01–D04), kode sudah terstruktur rapi — router, service, repository, schema masing-masing punya tanggung jawab sendiri. Tapi arsitektur yang bersih saja belum cukup. Tanpa test, tidak ada cara untuk membuktikan bahwa setiap komponen melakukan tugasnya dengan benar, apalagi setelah ada perubahan di masa depan.

Bayangkan jam tangan mekanik. Semua roda gigi terpasang, jam kelihatan jalan. Tapi bagaimana kamu tahu roda gigi nomor 3 berputar dengan rasio yang tepat? Kamu buka jam itu, lepas tiap komponen, dan test satu per satu — tanpa menunggu seluruh jam berputar. Itulah unit testing: test satu komponen, terpisah dari komponen lain.

2. Unit Test vs Integration Test

Perbedaan mendasar: unit test menguji satu unit logika secara terisolasi, sedangkan integration test menguji beberapa layer yang bekerja bersama. Hari ini fokus pada unit test.

Aspek	Unit Test	Integration Test
Scope	Satu fungsi / satu layer	Beberapa layer sekaligus
Server running?	Tidak perlu	Perlu (atau simulasi)
Database nyata?	Di-mock	Test DB
Kecepatan	Sangat cepat	Lebih lambat
Isolasi error	Mudah dilacak ke sumber	Error bisa dari mana saja
Kapan dijalankan	Setiap kali ada perubahan	Sebelum deploy / CI pipeline

3. Konsep Mock — Mengisolasi Dependensi

Setiap layer bergantung pada layer di bawahnya. Service bergantung pada repository. Router bergantung pada service. Kalau kita test service, kita tidak mau ikut-ikutan menjalankan repository nyata (yang butuh database). Solusinya: ganti dependensi dengan objek palsu yang kita kontrol penuh.

Analogi: kamu mau test mesin mobil. Kamu tidak perlu ban, setir, dan bodi untuk membuktikan mesin berputar. Kamu pasang mesin di test bench, sambungkan ke sumber bahan bakar palsu yang kamu kontrol, dan ukur output-nya. Mockito adalah 'test bench' itu.

MagicMock — return_value vs side_effect

Dua cara memberi instruksi ke mock, untuk dua kebutuhan yang berbeda:

Atribut	Perilaku	Kapan digunakan	Contoh
return_value	Selalu return nilai yang sama, berdasarkan apa yang sukses atau gagal	Mocking operasi yang sukses atau gagal	<code>mock.save.return_value = {'id': 1}</code>
side_effect (exception)	Melempar exception saat dipanggil	Singlisasi sistem rusak / error	<code>mock.save.side_effect = RuntimeError('DB down')</code>
side_effect (list)	Return elemen berbeda tiap kali dipanggil	Situasi yang membutuhkan mock get (side effect) dari data	<code>mock.get.side_effect = [data1, data2, None]</code>
side_effect (fungsi)	Jalankan fungsi custom setiap dipanggil	Setiap berbeda berdasarkan input yang diberikan	<code>mock.get.side_effect = lambda x: x*2</code>

4. Pytest Fixture — Setup yang Reusable dan Aman

Fixture adalah fungsi yang menyiapkan objek atau kondisi yang dibutuhkan test. Pytest akan inject fixture ke parameter fungsi test secara otomatis berdasarkan nama. Ini menghindari duplikasi setup di setiap test.

Pola dasar fixture:

```
@pytest.fixture
def mock_repo():
    return MagicMock() # dibuat fresh tiap test

@pytest.fixture
def service(mock_repo): # pytest inject mock_repo otomatis
    return ModelService(repo=mock_repo)
```

Fixture dengan Teardown (yield)

Kalau fixture perlu melakukan cleanup setelah test selesai — misalnya membersihkan `dependency_overrides` FastAPI — gunakan `yield`. Semua kode di bawah `yield` dijamin selalu dieksekusi oleh pytest, tidak peduli apakah testnya pass, fail, atau crash. Ini mencegah 'polusi' antar test.

Fixture dengan cleanup (yield pattern):

```
@pytest.fixture
def override_service(mock_service):
    app.dependency_overrides[get_model_service] = lambda: mock_service
    yield # test berjalan di sini
    app.dependency_overrides.clear() # selalu dieksekusi
```

Bahaya meletakkan `.clear()` di dalam body test: kalau test gagal sebelum baris itu, override tidak pernah dibersihkan. Test berikutnya menerima state yang salah — bug yang sangat sulit dilacak.

5. Tiga Unit Test yang Wajib Ada di Layered Architecture

5a. Schema Test (test_schema.py)

Schema adalah gerbang pertama. Test di sini membuktikan bahwa Pydantic benar-benar menolak input yang salah sebelum menyentuh service atau database. Tidak butuh mock, tidak butuh server — Pydantic adalah pure Python.

Apa yang dites	Hasil yang diharapkan	Kenapa penting
Input dengan tipe yang benar	Diterima, field terisi	Kontrak dasar schema
Input dengan tipe salah (string vs int)	raise ValidationError	Schema harus reject sebelum logika berjalan
Jumlah elemen kurang	raise ValidationError	Constraint domain (misal: harus 5 fitur)
Nilai infinity / NaN	raise ValidationError (custom validator)	Model ML tidak bisa handle inf/NaN
Nilai negatif (jika tidak valid)	raise ValidationError (custom validator)	Constraint bisnis (misal: umur tidak boleh negatif)

Pola test schema — valid dan invalid:

```
def test_valid_input_accepted():
    data = PredictionRequest(features=[0.5, 1.2, 3.1, 0.8, 2.4])
    assert len(data.to_feature_list()) == 5

def test_negative_value_raises():
    with pytest.raises(ValidationError):
        PredictionRequest(features=[1.0, -5000.0, 3.0, 4.0, 5.0])
```

5b. Service Test (test_service.py)

Service adalah tempat business logic hidup. Test di sini membuktikan bahwa logic-nya benar — tanpa peduli database nyata. Repository di-mock agar test bisa berjalan cepat dan terisolasi.

Apa yang dites	Hasil yang diharapkan	Kenapa penting
Input valid, repo mock return data	Service return hasil yang benar	Happy path — alur normal harus berjalan
Input invalid (list kosong, nilai 0)	raise ValueError	Service harus punya guard sendiri, tidak bergantung ke repo
Verifikasi interaksi: repo dipanggil	mock_repo.save.assert_called_once()	Membuktikan service benar-benar memakai repo
Repo raise RuntimeError	Service re-raise dengan __cause__	Error chain tidak boleh putus — penting untuk debug

Test error chain — cara yang benar:

```
def test_predict_handles_repo_error_and_preserves_cause(service, mock_repo):
    true_error = RuntimeError('Koneksi ke database terputus')
    mock_repo.save.side_effect = true_error

    with pytest.raises(RuntimeError) as excinfo:
        service.predict([1.0, 2.0, 3.0, 4.0, 5.0])

    assert excinfo.value.__cause__ == true_error
```

Kenapa tidak pakai match='pesan dari repo'? Karena itu justru membuktikan error BOCOR mentah dari repo.

Yang benar: service menangkap, re-raise dengan pesan baru, tapi __cause__ tetap menunjuk ke error asli.

from None vs from e: raise RuntimeError(...) from None menghapus __cause__ sepenuhnya. Test di atas akan GAGAL kalau service memakai from None, karena excinfo.value.__cause__ akan bernilai None bukan true_error.

5c. Endpoint Test (test_endpoint.py)

Endpoint adalah permukaan publik API. Test di sini membuktikan bahwa routing, status code, dan request/response contract berjalan benar. FastAPI menyediakan TestClient yang bisa kirim HTTP request tanpa menjalankan server — murni in-memory.

Apa yang dites	Hasil yang diharapkan	Kenapa penting
POST input valid	200 OK + body yang benar	Happy path endpoint
POST input invalid (tipe salah)	422 Unprocessable Entity	FastAPI+Pydantic auto-reject — tanpa mock
GET ID yang tidak ada	404 Not Found	Membuktikan logic 404 dari W11D04 benar
Verifikasi argumen ke service	mock.assert_called_once_with(..)	Endpoint harus lempar data yang tepat ke service

Dependency override — cara paling bersih test endpoint FastAPI:

```
# Setup: timpa dependency asli dengan mock
app.dependency_overrides[get_model_service] = lambda: mock_service

response = client.post('/predict', json={'features': [1.5,2.5,3.5,4.5,5.5]})

# Verifikasi argumen — bukan hanya status code
mock_service.predict.assert_called_once_with([1.5, 2.5, 3.5, 4.5, 5.5])

# Cleanup — selalu, atau pakai fixture yield
app.dependency_overrides.clear()
```

6. Panduan Lengkap — Kapan Pakai Apa

Referensi cepat untuk memutuskan tool, pattern, dan assertion yang tepat berdasarkan apa yang ingin kamu buktikan.

assert_called_once() dan variasinya

Method	Membuktikan	Kapan dipakai
assert_called_once()	Dipanggil tepat 1x (argumen tidak dicek)	Verifikasi interaksi terjadi, argumen tidak penting
assert_called_once_with(a, b)	Dipanggil tepat 1x dengan argumen a dan b	Verifikasi argumen yang masuk ke dependensi –
assert_called()	Dipanggil minimal 1x	Verifikasi efek samping terjadi
assert_not_called()	Tidak pernah dipanggil	Verifikasi cabang yang seharusnya tidak trigger –
call_count	Berapa kali dipanggil (integer)	Verifikasi tidak ada duplikasi call

pytest.raises — cara pakai dan variasinya

Pattern	Dipakai saat	Contoh
pytest.raises(ExcType)	Cukup pastikan exception terjadi	with pytest.raises(ValueError): fn()
pytest.raises(ExcType, match='...')	Cek tipe DAN pesan exception	match='Feature list tidak boleh kosong'
excinfo.value.__cause__	Verifikasi exception chain (from e) terjadi	assert excinfo.value.__cause__ == original_err
excinfo.value.__cause__ is None	Verifikasi chain DIPUTUS (from None) Jarang — biasanya kita ingin chain terjaga	

Kapan pakai mana: TestClient vs pure pytest

Kamu ingin membuktikan...	Gunakan	Catatan
Schema menolak input salah	pytest + ValidationError	Pure Pydantic, tidak butuh apapun
Service logic benar	pytest + MagicMock	Mock semua dependensi layer bawah
Endpoint return status code yang benar	TestClient	Mock service via dependency_overrides
Endpoint meneruskan data yang tepat	TestClient + assert_called_with	Verifikasi argumen, bukan hanya status
Seluruh stack berjalan end-to-end	TestClient + DB test nyata	Integration test — bukan unit test

7. Struktur File Test yang Disarankan

Lavout folder:

```
tests/
  conftest.py      <- shared fixtures (mock_repo, mock_model, client)
  test_schema.py   <- Pydantic validation tests
  test_service.py  <- business logic tests
  test_endpoint.py <- HTTP layer tests
  test_repository.py <- (nanti) dengan DB test
```

`conftest.py` — tempat fixture shared:

```
# tests/conftest.py
import pytest
from unittest.mock import MagicMock
from fastapi.testclient import TestClient
from app.main import app

@pytest.fixture
def mock_repo():
    return MagicMock()

@pytest.fixture
def client():
    return TestClient(app)
```

`conftest.py` dikenali `pytest` secara otomatis — tidak perlu diimport. Semua fixture di dalamnya tersedia untuk semua file test di folder yang sama.

8. Cara Menjalankan Pytest

Command	Efek
<code>pytest tests/ -v</code>	Semua test, verbose (tampilkan nama tiap test)
<code>pytest tests/test_service.py -v</code>	Hanya file tertentu
<code>pytest tests/ -k 'schema'</code>	Hanya test yang namanya mengandung 'schema'
<code>pytest tests/ -x</code>	Stop saat pertama kali ada test gagal
<code>pytest tests/ --tb=short</code>	Traceback ringkas — lebih mudah dibaca
<code>pytest tests/ -v --tb=short</code>	Kombinasi yang paling sering dipakai

9. Rangkuman Kunci

- Unit test = test satu komponen terisolasi. Tidak ada server, tidak ada database nyata.
- Mock menggantikan dependensi nyata. `return_value` untuk sukses, `side_effect` untuk error atau data dinamis.
- Fixture adalah setup yang reusable. `yield` di dalam fixture menjamin cleanup selalu terjal.
- `assert_called_once_with()` lebih kuat dari `assert_called_once()` — verifikasi argumen, bukan hanya bahwa method dipanggil.
- `excinfo.value.__cause__` membuktikan error chain (from `e`) terjaga. `from None` akan membuat test ini gagal — itu yang kita inginkan.
- `TestClient` memungkinkan test endpoint tanpa running server. `dependency_overrides` mengganti service nyata dengan mock.
- Bersihkan `dependency_overrides` lewat fixture `yield` — jangan di body test, karena tidak terjamin dieksekusi kalau test gagal.
- 422 tidak butuh mock — FastAPI + Pydantic menanganinya otomatis sebelum menyentuh service.

